

Report – BST with Go

Gerald Fattah

CS378H – Assignment 3

October 22nd, 2025

Overview:

The goal of this project was to detect which binary search trees (BSTs) constructed from different input lines are equivalent. Meaning they contain the same values and would produce the same BST structure when inserted in the same order. The program supports a sequential version and several parallel versions that use goroutines, channels, locks, and worker pools to speed up hashing and comparison on larger inputs.

Sequential Algorithm:

The baseline implementation reads each line of input, inserts the integers into a BST, and then performs these steps:

1. In-order traversal – Produces a sorted slice of values for each tree.
2. Hashing – The traversal output is turned into a SHA-1 hash.
3. Grouping by hash – Trees with different hashes are guaranteed to be different; only matching hashes are potential duplicates.
4. Deep comparison – For trees that share a hash, a pairwise comparison confirms whether the trees are truly equivalent.
5. Union-Find – Equivalence classes are formed so the final output can list groups of identical trees.

This version runs entirely in one thread and serves as the correctness reference before adding concurrency.

Hash Grouping:

After hashing, the results must be grouped in a shared map keyed by the hash, which introduces synchronization. I implemented two different approaches to handle this step. The first is a channel-collector design, where workers send (hash, id) pairs to a single goroutine that updates the map. This avoids locks entirely and guarantees correctness through serialization, but it can also become a bottleneck because one goroutine must process all updates. The second approach uses multiple workers that update the map directly while holding a mutex. This removes the single choke point and can scale better when many workers are running, although threads may occasionally stall while waiting for

the lock. On small inputs, the two strategies perform similarly, but on larger inputs the trade-off becomes clear: the channel version is simple and smooth until the collector saturates, while the mutex version spreads the work out more evenly at the cost of increased contention.

Parallel Hashing

Hashing is an embarrassingly parallel task because each tree can be traversed independently. I implemented two hashing modes controlled by a flag:

- goroutine-per-tree – Spawns one goroutine for each tree. This is great for coarse workloads, but spawning thousands of goroutines on fine workloads adds overhead.
- worker pool – Uses a fixed number of hashing goroutines and sends tree indices through a channel. This avoids excessive goroutine creation and produces more stable timing on fine workloads.

Parallel Comparison:

Once hash grouping is done, only trees in the same hash bucket need deeper comparison. I support:

- goroutine mode – spawns a goroutine per comparison
- worker-pool mode – uses a bounded buffer plus a fixed number of workers

On *simple.txt*, everything finishes so fast that overhead dominates. On *fine.txt*, the worker pool is noticeably more consistent because it reuses goroutines instead of spawning hundreds or thousands of them.

Both modes update the shared Union-Find structure to form final equivalence classes.

Results and Observations

- simple.txt – Tree sizes are tiny, and the work per task is so small that concurrency overhead dominates. Sequential and parallel versions look almost identical.
- coarse.txt – Larger trees make hashing and comparison expensive enough that concurrency pays off. Speedup increases up to around 4 workers before diminishing returns kick in.

- fine.txt – There are many small trees. Here, the worker-pool versions perform better and avoid goroutine spam. Both mutex and channel grouping behave similarly at scale since synchronization dominates.

- **Results**

===== RESULTS =====			
Input file: input/simple.txt			
DataMode	CompMode	HashWorkers	Time
channel	goroutine	1	701.7000 µs
channel	goroutine	2	1.4388 ms
channel	goroutine	4	733.3000 µs
channel	goroutine	8	0.0000 s
channel	pool	1	685.7000 µs
channel	pool	2	0.0000 s
channel	pool	4	0.0000 s
channel	pool	8	714.2000 µs
lock	goroutine	1	686.3000 µs
lock	goroutine	2	691.4000 µs
lock	goroutine	4	616.8000 µs
lock	goroutine	8	700.6000 µs
lock	pool	1	702.7000 µs
lock	pool	2	547.2000 µs
lock	pool	4	1.8787 ms
lock	pool	8	697.8000 µs

Input file: input/coarse.txt			
DataMode	CompMode	HashWorkers	Time
channel	goroutine	1	261.3911 ms
channel	goroutine	2	235.3372 ms
channel	goroutine	4	272.1734 ms
channel	goroutine	8	283.5530 ms
channel	pool	1	291.5078 ms
channel	pool	2	226.8271 ms
channel	pool	4	208.2377 ms
channel	pool	8	289.9116 ms
lock	goroutine	1	245.8095 ms
lock	goroutine	2	243.4066 ms
lock	goroutine	4	221.5878 ms
lock	goroutine	8	224.1447 ms
lock	pool	1	265.7617 ms
lock	pool	2	233.7993 ms
lock	pool	4	211.8038 ms
lock	pool	8	267.3377 ms

Input file: input/fine.txt

DataMode	CompMode	HashWorkers	Time
channel	goroutine	1	772.6701 ms
channel	goroutine	2	678.6044 ms
channel	goroutine	4	714.7180 ms
channel	goroutine	8	831.1271 ms
channel	pool	1	837.3118 ms
channel	pool	2	732.6413 ms
channel	pool	4	743.6583 ms
channel	pool	8	813.5739 ms
lock	goroutine	1	848.9405 ms
lock	goroutine	2	715.9175 ms
lock	goroutine	4	733.0701 ms
lock	goroutine	8	758.9362 ms
lock	pool	1	926.8854 ms
lock	pool	2	700.4283 ms
lock	pool	4	721.7020 ms
lock	pool	8	785.2449 ms

```
=====
channel goroutine 8 831.1271 ms
channel pool 1 837.3118 ms
channel pool 2 732.6413 ms
channel pool 4 743.6583 ms
channel pool 8 813.5739 ms
lock goroutine 1 848.9405 ms
lock goroutine 2 715.9175 ms
lock goroutine 4 733.0701 ms
lock goroutine 8 758.9362 ms
lock pool 1 926.8854 ms
lock pool 2 700.4283 ms
lock pool 4 721.7020 ms
lock pool 8 785.2449 ms
channel goroutine 8 831.1271 ms
channel pool 1 837.3118 ms
channel pool 2 732.6413 ms
channel pool 4 743.6583 ms
channel pool 8 813.5739 ms
lock goroutine 1 848.9405 ms
lock goroutine 2 715.9175 ms
lock goroutine 4 733.0701 ms
lock goroutine 8 758.9362 ms
lock pool 1 926.8854 ms
channel goroutine 8 831.1271 ms
channel pool 1 837.3118 ms
channel pool 2 732.6413 ms
channel pool 4 743.6583 ms
channel pool 8 813.5739 ms
lock goroutine 1 848.9405 ms
lock goroutine 2 715.9175 ms
lock goroutine 4 733.0701 ms
channel goroutine 8 831.1271 ms
channel pool 1 837.3118 ms
channel pool 2 732.6413 ms
```

Overall trend:

- Goroutines are fast, but not free.
- Worker pools shine on fine-grained workloads.
- Channels are simpler and cleaner, while mutexes avoid single-thread bottlenecks.

Takeaways:

This project demonstrated how algorithm structure interacts with Go's concurrency model. The BST equivalence problem breaks nicely into parallel stages, but the performance results show that *synchronization strategy matters more than just "adding more goroutines."* The best design depends on workload size:

- **Coarse workloads:** goroutine-per-task is perfectly fine
- **Fine workloads:** worker pools and controlled synchronization perform better

I also learned that channels and mutexes each have trade-offs rather than one being "better" in all cases.

Time Spent: 22 hrs